

EvoArena — Midterm Progress Report

Rohan Upadhyay | CSCI 411: Senior Seminar I | March 13, 2026

GitHub: <https://github.com/RyanX5/evoarena>

1. Project Overview

EvoArena is an agent-based evolutionary simulation where autonomous agents compete in a 2D arena and develop combat and survival behaviors through evolution. Each agent is controlled by a small neural network whose weights are shaped over generations using a genetic algorithm. Successful agents reproduce, passing their strategies to the next generation.

The intended use case is demonstrating how complex behaviors can emerge from simple agents under selection pressure, without any hand-coded rules. The project covers core CS concepts including neural computation, evolutionary algorithms, agent-based modeling, and system design.

2. Implementation Status

Since the initial proposal, all core systems have been implemented. The project has progressed from a foundation demo with randomly wandering agents to a fully operational neuroevolution simulation running multi-generation evolutionary loops.

Completed Modules

Module	Status	Description
<code>simulation/arena.py</code>	Complete	Arena environment, step loop, obstacle placement, agent management
<code>simulation/agent.py</code>	Complete	Neural-network-driven decisions, combat, fitness tracking
<code>simulation/neural_network.py</code>	Complete	8→12→4 feedforward NN, forward pass, weight serialization
<code>visualization/visualizer.py</code>	Complete	Real-time Pygame rendering, health bars, HUD
<code>evolution/evolve.py</code>	Complete	Elitist selection, Gaussian mutation, generation cycling
<code>main.py</code>	Complete	Generation loop, arena reset, visualizer integration

GitHub Repository

All source code, commit history, and documentation are available at:

<https://github.com/RyanX5/evoarena>

3. Timeline Comparison

The original proposal outlined the following schedule:

- **Week 1–2:** Arena and basic agent mechanics
- **Week 3–4:** Neural network control and combat
- **Week 5–6:** Evolutionary loop and fitness evaluation
- **Week 7–8:** Initial simulations and data collection
- **Week 9:** Metrics and analysis
- **Week 10:** Midterm demonstration and progress report

As of the midterm submission, all milestones through Week 9 have been completed on schedule. The simulation is running multi-generation evolutionary loops with observable behavioral changes across generations. No proposal adjustments have been required.

4. System Architecture

The project is organized into three independent modules, consistent with the original proposal:

- `simulation/` — agent logic, arena environment, neural network
- `visualization/` — real-time Pygame rendering
- `evolution/` — selection, mutation, generation cycling
- `config.py` — all tunable parameters in one place

The simulation engine operates completely independently of the visualizer. The arena can run headlessly for fast training, or render each frame in real time. This separation was a deliberate design decision made early in development and has proven important as the generation loop now creates a new arena instance each generation while the visualizer window persists.

5. Key Implementations Since Report 1

5.1 Neural Network Decision Making

In Report 1, agents used a placeholder `wander()` method for random movement. This has been replaced with a `decide()` method that builds an 8-element input vector from the environment and passes it through the neural network.

The input vector is defined as:

$$\mathbf{x} = \left[\frac{d_{enemy}}{d_{max}}, \frac{\theta_{enemy}}{\pi}, \frac{h_{self}}{H_{max}}, \frac{h_{enemy}}{H_{max}}, \frac{d_{wall}}{d_{max}}, \frac{v_x}{v_{max}}, \frac{v_y}{v_{max}}, 1.0 \right]$$

All inputs are normalized to $[-1, 1]$ before the forward pass. The network produces 4 outputs through two tanh layers:

$$\mathbf{h} = \tanh(W_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{o} = \tanh(W_2 \mathbf{h} + \mathbf{b}_2)$$

The first two outputs directly set the agent's velocity each step:

$$v_x = o_0 \cdot v_{max}, \quad v_y = o_1 \cdot v_{max}$$

```

def decide(self, agents: list, arena_width: int, arena_height: int):
    max_dist = np.sqrt(arena_width**2 + arena_height**2)
    enemies = [a for a in agents if a is not self and a.alive]

    nearest = None
    nearest_dist = float('inf')
    for enemy in enemies:
        d = np.sqrt((enemy.x - self.x)**2 + (enemy.y - self.y)**2)
        if d < nearest_dist:
            nearest_dist = d
            nearest = enemy

    if nearest is None:
        inputs = np.zeros(8)
        inputs[7] = 1.0
        outputs = self.brain.forward(inputs)
        self.vx = float(outputs[0]) * MAX_SPEED
        self.vy = float(outputs[1]) * MAX_SPEED
        return

    nearest_angle = np.arctan2(nearest.y - self.y, nearest.x - self.x)
    wall_dists = sorted([self.x, arena_width - self.x, self.y, arena_height - self.y])
    corner_exposure = wall_dists[0] + wall_dists[1]
    wall_penalty_score = max(0.0, 1.0 - (corner_exposure / (config.WALL_DISTANCE * 2)))
    self.steps_near_wall += wall_penalty_score
    nearest_wall_dist = wall_dists[0]

```

```

inputs = np.array([
    nearest_dist / max_dist,
    nearest_angle / np.pi,
    self.health / MAX_HEALTH,
    nearest.health / MAX_HEALTH,
    nearest_wall_dist / max_dist,
    self.vx / MAX_SPEED,
    self.vy / MAX_SPEED,
    1.0
])

outputs = self.brain.forward(inputs)
self.vx = float(outputs[0]) * MAX_SPEED
self.vy = float(outputs[1]) * MAX_SPEED

```

5.2 Combat System

Agents now engage in combat. Each step, an agent checks all living enemies within `ATTACK_RANGE` pixels and deals damage to each. Kill credit is assigned to the attacker when an enemy's health reaches zero.

```

def attack(self, agents: list):
    enemies = [a for a in agents if a is not self and a.alive]
    in_range = [a for a in enemies if
        np.sqrt((a.x - self.x)**2 + (a.y - self.y)**2) <= ATTACK_RANGE]
    for enemy in in_range:
        enemy.take_damage(ATTACK_DAMAGE)
        self.damage_dealt += ATTACK_DAMAGE

```

```
if not enemy.alive:
    self.kills += 1
```

5.3 Fitness Scoring

Each agent accumulates a fitness score throughout its lifetime based on four weighted components:

$$F = w_{dmg} \cdot D + w_{kills} \cdot K + w_{steps} \cdot S - w_{wall} \cdot W$$

Where D is total damage dealt, K is kill count, S is steps survived, and W is the accumulated wall proximity penalty. The weights are:

$$w_{dmg} = 2.0, \quad w_{kills} = 150.0, \quad w_{steps} = 1.0, \quad w_{wall} = 2.0$$

```
def fitness(self) -> float:
    w_dmg, w_kills, w_steps, w_near_wall = 2.0, 150.0, 1.0, 2.0
    return (
        self.damage_dealt * w_dmg +
        self.kills * w_kills +
        self.steps_alive * w_steps -
        self.steps_near_wall * w_near_wall
    )
```

Kill reward is weighted highest at 150 points to incentivize aggressive behavior over passive survival. The wall penalty was introduced after observing reward hacking behavior (described in Section 6).

5.4 Evolution Engine

At the end of each generation, the top 20% of agents by fitness are selected as elites. The new population is filled by randomly sampling elites and applying Gaussian mutation to their weights:

$$\mathbf{w}' = \mathbf{w} + \mathcal{N}(0, \sigma^2 \mathbf{I}), \quad \sigma = 0.1$$

```
def next_generation(agents: list) -> list:
    agents.sort(key=lambda a: a.fitness(), reverse=True)
    elite_count = max(1, int(len(agents) * 0.2))
    elites = agents[:elite_count]

    new_agents = []
    while len(new_agents) < config.POPULATION_SIZE:
        elite = random.choice(elites)
        weights = elite.brain.get_weights()
        mutated = weights + np.random.normal(0, config.MUTATION_RATE, size=weights)
        new_brain = NeuralNetwork(weights=mutated)
        new_agents.append(Agent(x=0, y=0, brain=new_brain))

    return new_agents
```

6. Challenges and Solutions

6.1 Reward Hacking — Wall Hugging

The most significant challenge encountered was reward hacking. In early generations, agents discovered that pressing against arena walls minimized

exposure to enemies and maximized survival time. Since survival time was the dominant fitness component, wall hugging became the evolutionarily stable strategy by generation 10–15.

Solution: A wall proximity penalty W was introduced. Rather than a binary threshold, the penalty accumulates each step proportionally to corner exposure — the sum of the two smallest wall distances d_1 and d_2 :

$$p_t = \max \left(0, 1 - \frac{d_1 + d_2}{2 \cdot d_{threshold}} \right), \quad W = \sum_t p_t$$

This makes the penalty continuous — agents closer to walls and corners accumulate more penalty, with no hard boundary to exploit. Since $w_{wall} = 2.0 > w_{steps} = 1.0$, wall proximity is net negative even accounting for survival time gained.

6.2 Corner Exploitation

After an initial binary wall penalty was introduced, agents adapted within a few generations by locating the exact boundary of the penalty zone and positioning at the threshold distance from two walls simultaneously. By generation 16, the entire population had converged to sitting at exactly 80px from the bottom and left walls.

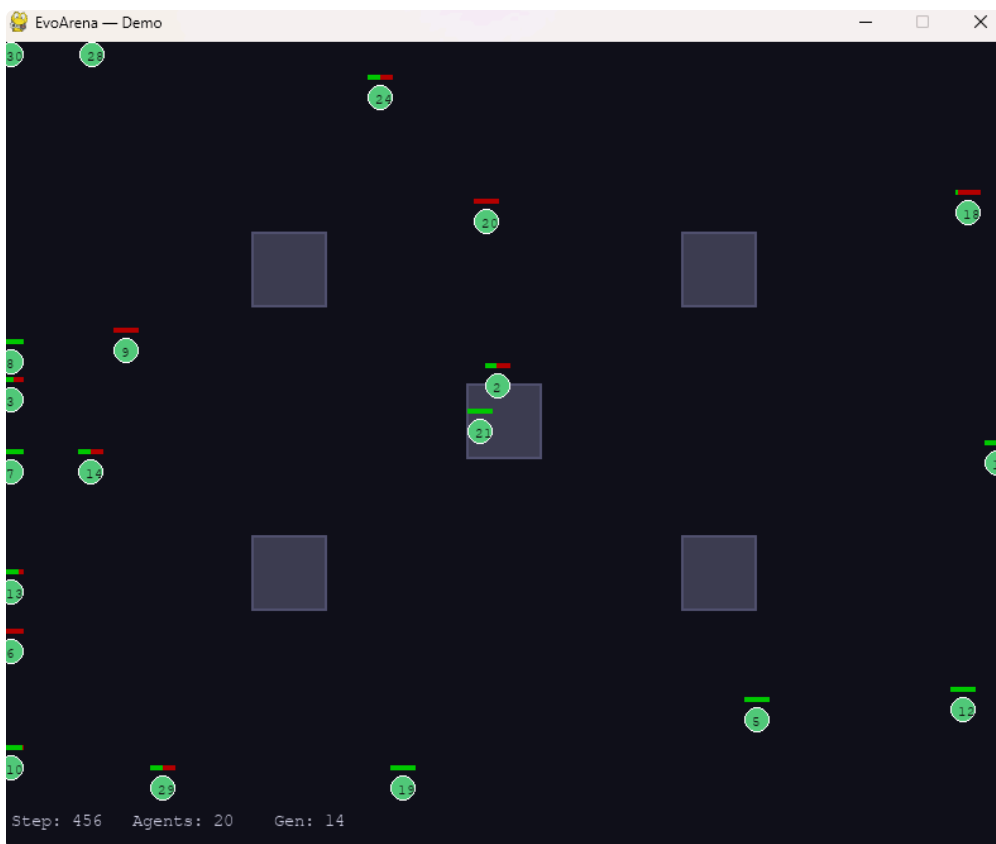


Fig 1: Generation 15 — agents converge to exactly 80px from walls, exploiting the hard penalty threshold

This is a clear example of reward hacking where agents optimize the letter of the fitness function rather than its intent. The gradual corner exposure penalty described in Section 6.1 resolved this by removing the exploitable hard boundary.

6.3 Population Diversity

With a population of 10 and 20% elitism, only 2 agents were selected as elites per generation. This caused rapid convergence where all agents were nearly identical mutations of the same two elites. Population size was increased to 30, providing 6 elites per generation and significantly more behavioral diversity.

7. Demonstration

A recorded demonstration of the simulation running from generation 1 through generation 60 is available at:

https://www.youtube.com/watch?v=BHfS_mRVInA

The following screenshots document the key behavioral stages observed across generations.



Fig 2: Generation 1 — agents moving erratically based on random initial weights

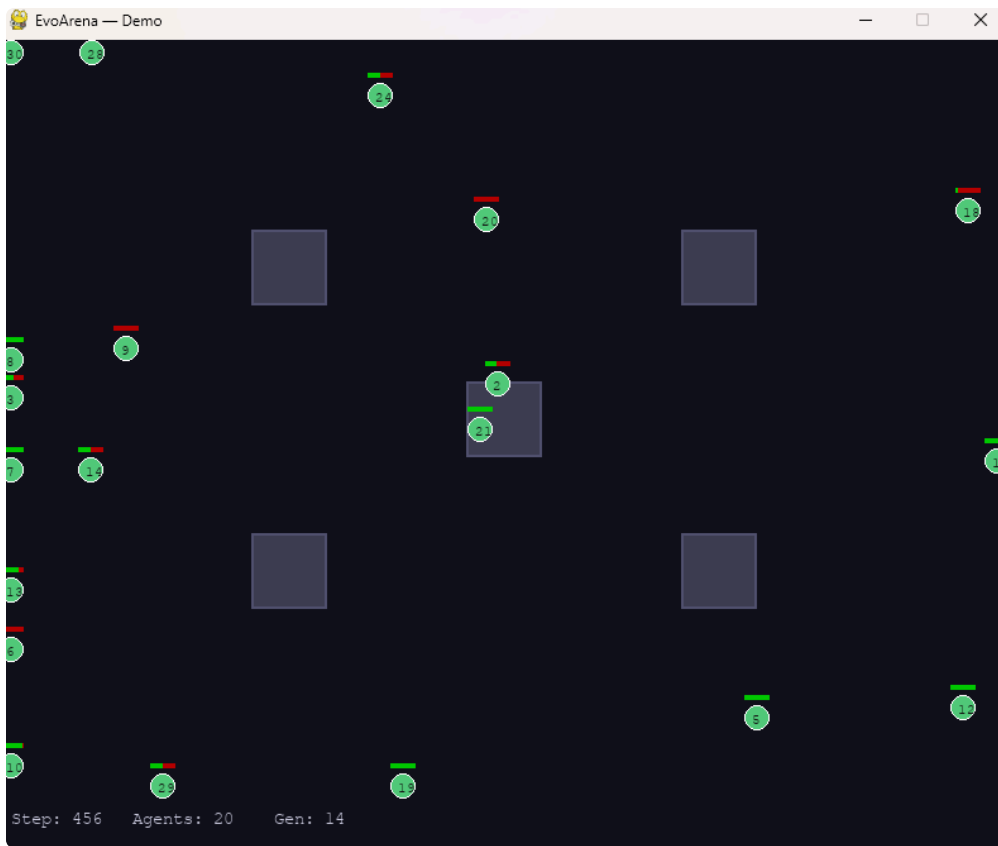


Fig 3: Generation 15 — wall hugging emerges as the dominant strategy

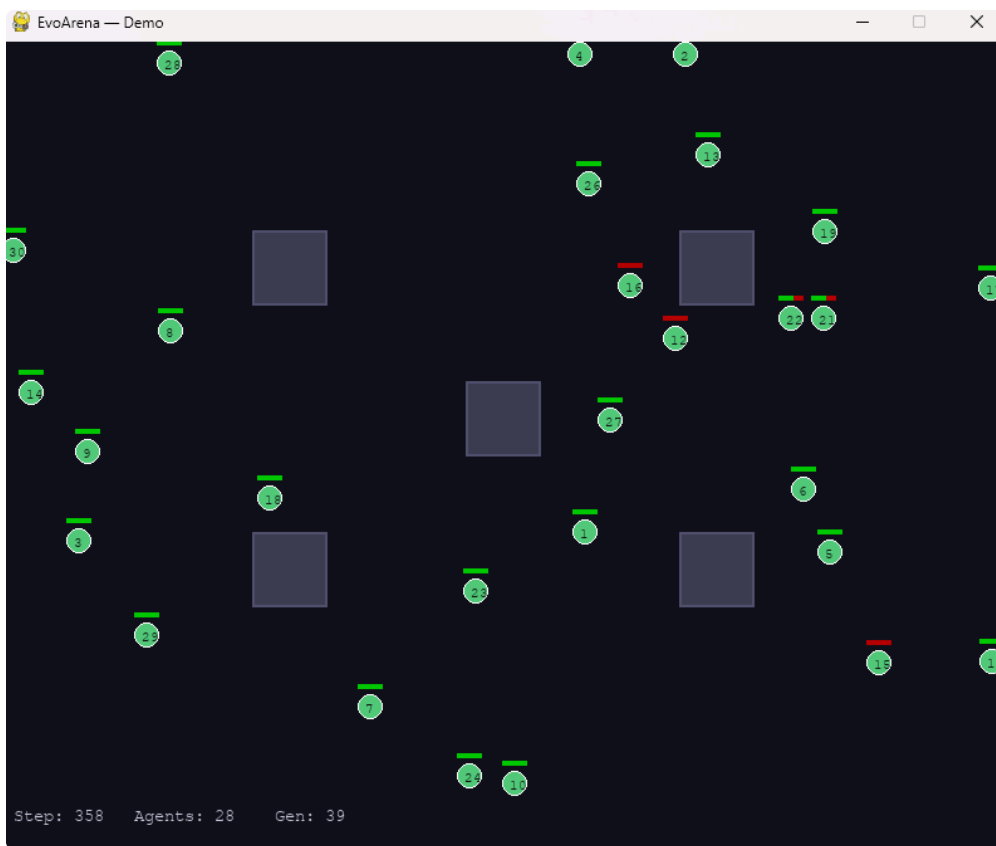


Fig 4: Generation 39 — agents begin clustering toward the center following fitness function adjustments

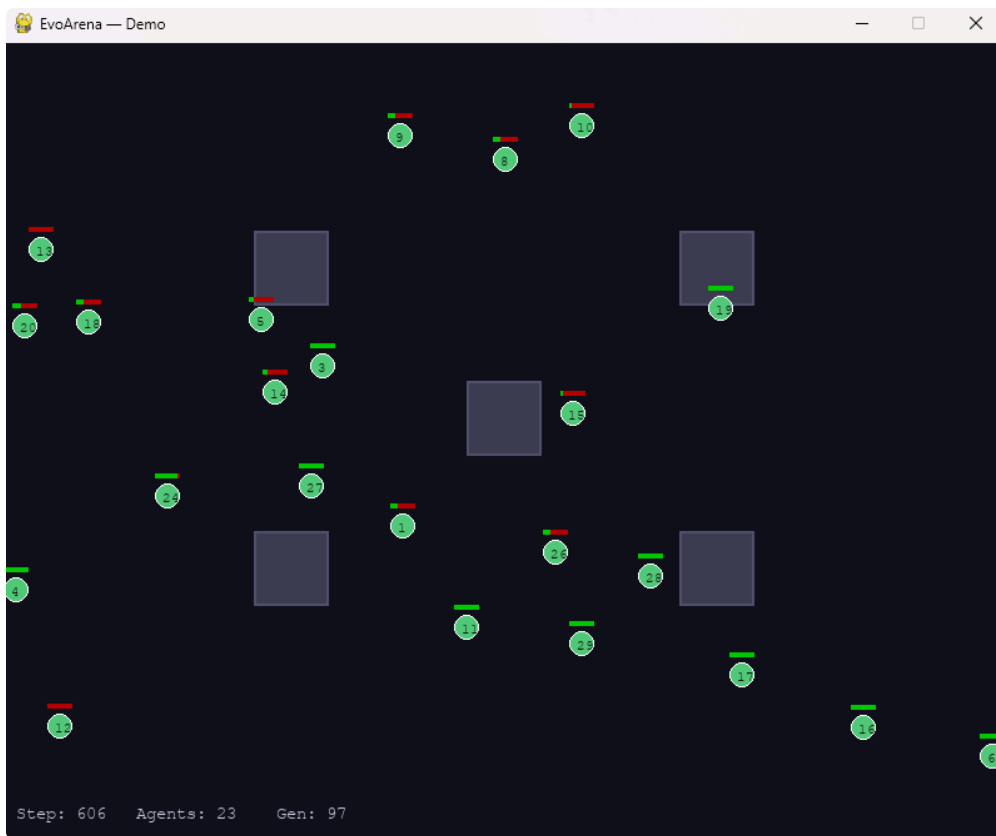


Fig 5: Generation 50+ — strategy divergence: passive sitters coexist with aggressive hunters

The full behavioral arc — random chaos → wall hugging → corner exploitation → center clustering → strategy divergence — demonstrates that the evolutionary loop is functioning correctly and agents are meaningfully adapting to selection pressure across generations.

8. Future Plan for Final Delivery

Remaining Technical Milestones

Week 11–12:

- Wire up the remaining two NN outputs (attack intent, flee mode) to enable more nuanced behavioral decisions
- Implement obstacle collision detection so agents interact physically with arena obstacles
- Add fitness graph visualization — plot average and best fitness per generation in real time
- Implement crossover: mixing weights from two elite parents to create children, in addition to mutation

Week 13:

- Parameter sensitivity analysis — systematically vary mutation rate, population size, and fitness weights and document effects on emergent behavior
- Run extended simulations (100+ generations) and document behavioral trends
- Save and replay champion agents from different generations for comparative analysis

Week 14:

- Final written report covering system design, algorithms, experiments, and results
- Polish GitHub repository — clean README, inline documentation, reproducibility instructions
- Final video presentation demonstrating multi-generation behavioral evolution

Final Deliverables

- Fully operational evolutionary simulation with combat, neural control, and multi-generation evolution
 - Fitness trend visualization across generations
 - Parameter analysis comparing fitness function variants
 - Complete GitHub repository with source code and documentation
 - Final written report
 - Final video demonstration
-

9. AI Tool Acknowledgment

AI tools were used to assist with minor code suggestions, debugging, and documentation drafting throughout this project. All AI-assisted outputs were reviewed, modified, and integrated with full understanding of the implemented algorithms. This is documented in the GitHub repository README as required.